MLVU information sheet

Please include this page in your report either at the start or at the end, before the appendix. Do not change the formatting.

Group number

156

Authors

name	student number
Conrad Clough	2699315
Samuel Power	2800107
ANDI FILL THIS OUT :)	THIS TOOOOO

Software used *Describe briefly which software and libraries you relied on. If you built a particular algorithm from scratch, make sure to state it explicitly here.*

We used Jupyter, numpy, pandas, matplotlib, and sklearn. Jupyter was used for ease of modifying our program without needing to redo data processing; numpy, pandas, and matplotlib were used for interacting with data and displaying results; sklearn was used for their model implementations.

Use of AI tools *If you used tools like ChatGPT or Github Copilot in any phase of the report, please detail here which tools you used, what parts of the work you used them for and how. Note that if you use an AI tool and don't report it, it's considered fraud, so be detailed. Note also that having an AI tool write for you is not allowed. If no AI tools were used, please state that explicitly.*

- Conrad: ChatGPT was used to generate the code that created our graphs, specifically the styling.

Link to code (optional) *If you would like to share your code, you can link to your repository here.*

Group disagreements (optional) *If there are any disagreements in the group, you may* ***not*** *remove a student from the author list without their consent. You should bring disagreements to our attention early. As a last resort, you can describe any grievances here.*

How can machine learning-based DDoS detection models be designed to minimise false positive rates while maintaining high detection accuracy on both low- and high-powered hardware?

Conrad Clough
VUNetId: cch250
Student Nr: 2699315
c.j.clough@student.vu.nl

Samuel Power
VUNetId: spo237
Student Nr: 2800107
s.j.power@student.vu.nl

mr. andy
VUNetId: x
Student Nr: x
lovelyman@student.vu.nl

I. Abstract

Distributed Denial of Service (DDoS) attacks continue to disrupt network services, causing downtime and economic damage. Existing techniques for mitigating this often struggle to whatever they struggle with, find a paper and cite it. This paper investigates the design of DDoS detection models using machine learning. We evaluate three classifiers, k-Nearest Neighbours (kNN), Logistic Regression (LR), and Random Forests, by analysing their accuracy, false positive rates, and resource usage under controlled conditions. Our results indicate that whatever we find. One sentence about each, what they do well and what they do bad. These findings suggest that model selection is dependent on hardware specifications and the relative importance of minimising false positives, providing a practical guide for using machine learning-based DDoS detection in diverse contexts.

Keywords: DDoS detection, machine learning, false positive rate, k-Nearest Neighbours, Random Forest, Logistic Regression, computational efficiency

II. Introduction

The proliferation of online services has made networks increasingly vulnerable to Distributed Denial of Service (DDoS) attacks, which can overwhelm servers and disrupt them, causing significant operational downtime and economic damage. Detecting these attacks efficiently is challenging because high accuracy must be balanced with low false positive rates; denying service to legitimate users is highly undesirable, as is allowing too much malicious traffic through. Machine learning offers a promising alternative to traditional, rule-based approaches; patterns in network traffic can be used to identify evolving malicious activity in real time. This document investigates the design of machine learning-based DDoS detection models with an emphasis on maintaining high detection accuracy while reducing false positives, and evaluates their resource usage across a range of hardware to ensure usability for a variety of situations. Specifically, k-Nearest Neighbours (kNN), Logistic Regression (LR), and Random Forest classifiers are compared in terms of both accuracy, false positive rate, and performance.

Blah blah blah. We can quote code `like this`. We can also reference sections like this Section II or even define labels on things like figures and images and reference them later.

Wanna reference something in the bibliography? Just do it with another label [1]

The report is structured as follows:

- In Section III we consider existing methods of DDoS detection.
- In Section VII we discuss the dataset we used and how it was prepared for use in training.
- In Section VIII we explain our classifiers and how we ran our experiments.
- In Section IV we show our results.
- In Section V we interpret our results and address what they mean.
- In Section VI we summarise our findings.

III. Related Work

The algorithms in this project were trained on data obtained from kaggle (MAYBE NEED TO LINK HERE??). The data was already mostly equally balanced between benign traffic and DDoS traffic. The data was also shuffled in the event that some unknown order had been preserved upon creation. Notably though the data is quite large and does not fit into the memory of the hardware of the researchers. For this reason, the project employs a chunking approach, so that the data can be fully trained upon. The data is read into the models in chunks. Because of this, the entire dataset is not loaded in at once as not all data points are used. Balancing is done to ensure that training is not hindered by biases. This could for example happen when a model favours the majority class, when trained on imbalanced data. This ensures accurate, fair, and reliable predictions across all classes. In this case, 100 000 rows for DDoS and 100 000 rows for benign traffic were used. The features were also modified. The SimpleImputer function was used to replace cells with no value safely. Some features were also dropped; namely the identifying columns and other non-revelvant determined data were all dropped. The dropped columns were as follows:

Column
`Label`
`Unnamed: 0`
`Flow ID`
`Src IP`
`Dst IP`
`Timestamp`
`Src Port`
`Dst Port`

The features of KNN and LR were also scaled because they depend on distances and coefficient magnitudes. This was not necessary for RF because tree splits are threshold-based and largely scale-invariant. The scaling is important because it ensures that features with larger numerical ranges do not disproportionately influence the model's behavior.

The protocol was additionally one-hot encoded since it is categorical. The data was also cleaned, where inf values were relaced with NaN. For robustness, outliers were identified through use of the median instead of the mean. In terms of training the models, a 60/20/20 split was employed where 60% of the data was used for training, 20% for validation and 20% for testing.

The models that will be evaluated are a k-Nearest Neighbours model (kNN), a random forest model (RF), and a logistical regression (LR) model. Each of these models has tuned hyperparameters. A variety of possible hyper-parameters are tested, and the best one is chosen by the argmax function on the validation set's accuracy. To make sure that the best hyper-parameters aren't simply overfitting the models will also be benchmarked to determine the extent of over or underfitting.

The correctness of the models will be evaluated through the accuracy of the model upon the test data. This mainly

takes into account the extent to which false positives were minimised while true positives were maximised. The project will make use of confusion matrices for this. Since these are binary classification models, the receiver operating characteristic (ROC) curve as well as the area under the curve (AUC) will be used to jointly evaluate the extent to which these models are accurate. While ROC is commonly used for datasets that are imbalanced, the project employs upon the already balanced dataset as a metric to be included for the overfitting analyses, which will determine the strengh of the models with respect to the aforementioned research question.

The research question scope includes the use of models on low and high powered hardware. Correctness remains the same as previously defined, with the constraint that the models must run in [XXXXXXXX][TIMEFRAME] to qualify as having successfully run on the specified hardware. To evaluate this, the high powered hardware will encompass a modern computer who's computation power is at least X Y Z, and a low-powered computer like a raspberry-pi who's computation power is maximally X Y Z. In place of a physical raspberry-pi, a raspberry-pi emulator will be used. **WILL BE MEASURUED IN FLOPS**

To strengthen the future findings the report will also make use of additional metrics.

1) Random Forest will make use of the feature importance that is a property of tree-based models. This will assist in the verification that the model's results on testing data are not correlated to a data anomaly that would not allow the model to continue to maintain it's accuracy scores in real world deployment.
2) The logistical regression model will make use of threshold tuning to allow the authors to trade recall for precision in a controlled way, which could help in reducing false positives. This is important as while both high recall and high precision are important, it is also very important that no real email lands in spam. Perhaps moreso than a spam email landing in the normal inbox.

Add still: how is correctness evaluated? MORE DETAILED, like precision, recall, ROC/AUC etc. feature scaling in KNN and LR (not present in RF), overfitting analysis details, feature importance for RF and why we do that, threshold tuning in LR only, use of SimpleImputer for cleaning, complete hardware XXXX placeholders

– cite 10 papers

[W I L L | F I N I S H | A N D | F O R M A T | S O O N -SAMUEL]

– new kaggle dataset run on thing – Hypotheses - time complexities, hardware

IV. Results

V. Discussion

VI. Conclusion

knn does worse on imbalanced because the larger group sizes 'pull' knn into them

TABLE I
Resource usage for model training and inference

Model	RAM Δ (MB)	CPU Time (s)	Wall Time (s)
kNN (training)	2034.49	10121.29	1163.05
kNN (inference)	60.95	1402.22	141.90
kNN (inference, Raspberry Pi)	290.60	5344.74	1408.22
Random Forest (training)	2358.29	1619.48	116.39
Random Forest (inference)	6.48	17.46	7.95
Random Forest (inference, Raspberry Pi)	1464.93	51.85	65.14
Logistic Regression (training)	233.83	69.41	42.31
Logistic Regression (inference)	36.73	9.42	7.42
Logistic Regression (inference, Raspberry Pi)	1632.62	27.53	30.78

find a paper about this

discuss how you can change this, with k tuning or with weights on the classes that represent the imbalance

reference here asdasd [2], [3], [4]

VII. Dataset and Preprocessing

Three models were trained on a dataset consisting of network traffic, balanced between DDoS traffic and benign traffic. These models were kNN, logistic regression, and random forest. The data was extracted from public IDS datasets, produced in different years and with different techniques of generating traffic. There are 12 794 627 datapoints and 84 features. The models' inference abilities were then tested on an identically formed dataset that was unbalanced, with 20% DDoS traffic and 80% benign. Implementation of each model was provided by the `sklearn` library for Python.

In order to manage resource and time constraints, training and inference were limited to 200 000 labels. This data was then put into a `pandas` dataframe for efficiency and ease of management. Particular features that were either irrelevant or non-numeric were also dropped - both kNN and logistic regression require numeric features. An alternative way of tackling non-numeric features is to one-hot encode them, where categoric features are given an integer identifier. This massively increases the memory requirements of the model and was therefore decided against, with the exception of encoding the `Protocol` feature which was the only relevant categorical feature present. This resulted in 79 features.

The next step was cleaning; all features were represented with a 64-bit float and invalid numbers were cast to NaN for predictable management.

Next, the dataset was split into 60% training, 20% validation, and 20% test data. The training set was used for fit the model parameters, and was the largest to allow the models to effectively capture patterns in the data. The validation set was used for hyperparameter tuning; as the models haven't

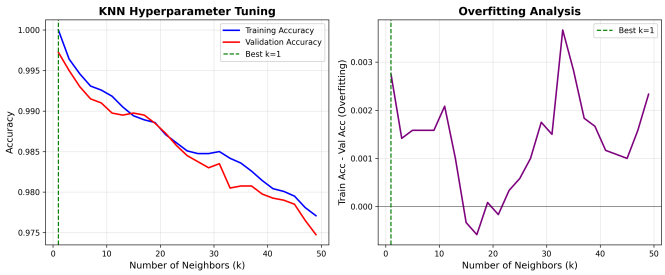

Fig. 1. Tuning hyperparameters for kNN for accuracy.

encountered the data before this is a suitable playground for adjustment. The final test set allows finalisation of ideal hyperparameters, and an additional verification that they are indeed ideal without any bias.

Finally, features were then scaled with a mean of 0 and a standard deviation of 1, which is z-score normalisation. This is because logistic regression is sensitive to feature magnitude as it uses gradient descent which can struggle when, for example, one feature takes values $[0, 1]$ and another takes $[2^{32}, 2^{64}]$. This prevents particular features completely dominating calculations and having inflated importance. It also allows faster convergence to the gradient's local minima. This scaler was applied to the training data, and then used to transform all data to make it homogeneous for the models. This is the last step to prepare the data.

Exact implementation details can be viewed at the codebase's git repository [1].

VIII. Methodology

A. kNN

The first model evaluated was kNN. This works by finding the k-nearest points in the training dataset and uses their class to predict the class of nearby datapoints. For 79 features, the feature space has a dimensionality of $\mathbb{R}^{79}$ which makes visualisation challenging, but the idea of 'nearby datapoints' generlises to high-dimension Euclidean spaces and therefore kNN is perfectly usable, though potentially slow REPHRASE, CITE, OR REFERENCE HYPOTHESIS. Varying hyperparameters k were tested to maximise accuracy and minimise overfitting, see Fig. 1. Larger k can lead to overfitting and smaller k can lead to underfitting, so finding the balance was important.

After the optimal hyperparemter k was chosen, 5-fold cross-validation was used, wherein the validation and training sets were combined and split into 5 sections. This allows for addressing the tradeoff between bias and variance, finding a model that maximises the benefit gained for the best inference results. Finally, the model was trained and then evaluated a single time on the test data. Using the test data a single time to evaluate final performance is extremely important to prevent provide an independent analysis of the model's performance - repeated use of the test data means this is no longer an evaluation on unseen data and therefore valueless.

B. Logistic Regression

C. Random Forest

Once the models were evaluated, we created confusion matrices, ROC curves, and computed AUC scores for each of our models. The models themselves were also saved to later be used for inference. This inference was performed with identical dataset preprocessing on the unbalanced dataset. Confusion matrices, ROC curves, and AUC scores were also computed for the inference run, and only one run was performed per model as the results are deterministic.

Throughout both training and analysis, performance statistics were collected. These were RAM usage, CPU time, and wall clock time. Training was done on a desktop computer equipped with a 12th Gen Intel(R) Core(TM) i7-12700K and 32GiB of DDR5 4800MHz RAM, and inference was ran on both the same desktop computer and a Raspberry Pi 4 with a Quad core Cortex-A72 (ARM v8) 64-bit SoC @ 1.8GHz and 4GiB of DDR4 3200MHz RAM..

The algorithms in this project were trained on data obtained from kaggle (MAYBE NEED TO LINK HERE??). The data was already mostly equally balanced between benign traffic and DDoS traffic. The data was also shuffled in the event that some unknown order had been preserved upon creation. Notably though the data is quite large and does not fit into the memory of the hardware of the researchers. For this reason, the project employs a chunking approach, so that the data can be fully trained upon. The data is read into the models in chunks. Because of this, the entire dataset is not loaded in at once as not all data points are used. Balancing is done to ensure that training is not hindered by biases. This could for example happen when a model favours the majority class, when trained on imbalanced data. This ensures accurate, fair, and reliable predictions across all classes. In this case, 100 000 rows for DDoS and 100 000 rows for benign traffic were used. The features were also modified. The SimpleImputer function was used to replace cells with no value safely. Some features were also dropped; namely the identifying columns and other non-revelvant determined data were all dropped. The dropped columns were as follows:

Column
`Label`
`Unnamed: 0`
`Flow ID`
`Src IP`
`Dst IP`
`Timestamp`
`Src Port`
`Dst Port`

The features of KNN and LR were also scaled because they depend on distances and coefficient magnitudes. This was not necessary for RF because tree splits are threshold-based and largely scale-invariant. The scaling is important because it ensures that features with larger numerical ranges do not disproportionately influence the model's behavior.

The protocol was additionally one-hot encoded since it is categorical. The data was also cleaned, where inf values were relaced with NaN. For robustness, outliers were identified through use of the median instead of the mean. In terms of training the models, a 60/20/20 split was employed where 60% of the data was used for training, 20% for validation and 20% for testing.

The models that will be evaluated are a k-Nearest Neighbours model (kNN), a random forest model (RF), and a logistical regression (LR) model. Each of these models has tuned hyperparameters. A variety of possible hyper-parameters are tested, and the best one is chosen by the argmax function on the validation set's accuracy. To make sure that the best hyper-parameters aren't simply overfitting the models will also be benchmarked to determine the extent of over or underfitting.

The correctness of the models will be evaluated through the accuracy of the model upon the test data. This mainly takes into account the extent to which false positives were minimised while true positives were maximised. The project will make use of confusion matrices for this. Since these are binary classification models, the receiver operating characteristic (ROC) curve as well as the area under the curve (AUC) will be used to jointly evaluate the extent to which these models are accurate. While ROC is commonly used for datasets that are imbalanced, the project employs upon the already balanced dataset as a metric to be included for the overfitting analyses, which will determine the strengh of the models with respect to the aforementioned research question.

The research question scope includes the use of models on low and high powered hardware. Correctness remains the same as previously defined, with the constraint that the models must run in [XXXXXXXX][TIMEFRAME] to qualify as having successfully run on the specified hardware. To evaluate this, the high powered hardware will encompass a modern computer who's computation power is at least X Y Z, and a low-powered computer like a raspberry-pi who's computation power is maximally X Y Z. In place of a physical raspberry-pi, a raspberry-pi emulator will be used. **WILL BE MEASURUED IN FLOPS**

To strengthen the future findings the report will also make use of additional metrics.

1) Random Forest will make use of the feature importance that is a property of tree-based models. This will assist in the verification that the model's results on testing data are not correlated to a data anomaly that would not allow the model to continue to maintain it's accuracy scores in real world deployment.
2) The logistical regression model will make use of threshold tuning to allow the authors to trade recall for precision in a controlled way, which could help in reducing false positives. This is important as while both high recall and high precision are important, it is also very important that no real email lands in spam. Perhaps moreso than a spam email landing in the normal inbox.

Add still: how is correctness evaluated? MORE DETAILED, like precision, recall, ROC/AUC etc. feature scaling in KNN and LR (not present in RF), overfitting analysis details, feature importance for RF and why we do that, threshold tuning in LR only, use of SimpleImputer for cleaning, complete hardware XXXX placeholders

– cite 10 papers

[W I L L | F I N I S H | A N D | F O R M A T | S O O N -SAMUEL]

– new kaggle dataset run on thing – Hypotheses - time complexities, hardware

References

[1] P. S. Clough Conrad and A. Stolle, "Project Codebase on Github." [Online]. Available: https://github.com/Flootersmm/ml_project

[2] S. K. S. P. Y. e. a. Rajendran R., "Detection of DoS attacks in cloud networks using intelligent rule based classification system.," *Cluster Computing*, vol. 22, no. ?_number, pp. 423–434, Feb. 2019, doi: 10.1007/s10586-018-2181-4.

[3] S. Dong and M. Sarem, "DDoS Attack Detection Method Based on Improved KNN With the Degree of DDoS Attack in Software-Defined Networks," *IEEE Access*, vol. 8, no. , pp. 5039–5048, 2020, doi: 10.1109/ACCESS.2019.2963077.

[4] A. I. Gide and A. A. M. zu, "A Real-Time Instrusion Detection System for DoS/DDoS Attack Classification in IoT Networks Using KNN-Neural Network Hybrid Technique," *Babylonian Journal of Internet of Things*, vol. 2024, no. , pp. 60–69, 2024, doi: 10.58496/BJIoT/2024/008.